

자기소개

TypeScript와 Next.js를 기반으로 사용자 화면을 구현하고 서비스 구조를 개선해 온 **2년 4개월 경력의 프론트엔드 개발자**입니다.

실무에서는 Next.js App Router 마이그레이션과 디자인 시스템 구축을 진행한 경험이 있고,
개인 프로젝트에서는 OpenAPI 기반 API 타입 자동화와 Docker·AWS CI/CD 및 AI 협업 개발 체계를 직접 설계하고 검증했습니다.

반복되는 UI와 복잡한 데이터 흐름을 정리해 팀내에서 **일관성있고 유지보수에 용이한 구조로 만드는 데** 관심이 있습니다.
기능을 구현한 뒤에는 그 화면뿐 아니라 연관된 부분도 같이 기능이 의도대로 동작하는지 직접 확인하며 맡은 범위를 마무리합니다.

주요 경험

1. 세 도메인의 컴포넌트를 통합한 디자인 시스템 구축

B2B·B2C·어드민의 버튼, 토틱, 드롭다운의 사용처와 예외를 조사하고 공통 규격을 정해 기존 화면 전체에 적용했습니다.

2. 기존 동작을 유지한 Next.js App Router 마이그레이션

스کیل업 서비스의 목록·상세·내 강의 화면을 Pages Router에서 App Router로 전환하고 공통 네비게이션 가드를 재구성했습니다.

3. OpenAPI·openapi-typescript 기반 API 타입 자동화

NestJS 코드에서 OpenAPI 명세를 생성하고 openapi-typescript를 활용하여 프론트엔드 타입과 TanStack Query 요청 계층을 자동화했습니다.

4. Turborepo 모노레포와 Docker·AWS·GitHub Actions 기반 CI/CD 구축

프론트엔드와 백엔드를 모노레포와 Docker 이미지로 구성하고 ECR·EC2·RDS·S3를 연결한 배포 흐름을 구축했습니다.

업무 태도

1. 반복되는 문제를 구조적으로 개선하는 개발자입니다.

디자인 시스템 구축에서는 컴포넌트별 세 도메인의 사용처와 예외를 조사해 공통 규격으로 통합할 영역과 별도로 분리할 영역을 나눴습니다.

관리자 레거시에서는 데이터 흐름을 한 번에 바꾸지 않고 작업중인 화면부터 순차적으로 SWR로 전환했습니다.

2. 현재 서비스와 팀의 제약을 고려해 기술의 적용 범위를 판단하는 개발자입니다.

App Router 전환에서는 새로운 기능을 바로 적용하기보다 기존 화면의 동작을 유지하는 것을 우선했습니다.

API 문서 불일치 이슈에서는 tRPC, 타입 패키지 공유, OpenAPI 스펙 활용 방식을 비교하고 현재 백엔드 구조와 운영 부담 및 리소스를 고려해 적용 가능한 방식을 선택했습니다.

3. 새로운 기술을 직접 검증하고 업무에 연결하는 개발자입니다.

Turborepo와 Docker 및 AWS와 GitHub Actions를 직접 구성하고 OpenAPI와 openapi-typescript를 활용한 API 타입 자동화를 구현했습니다.

검증한 OpenAPI 방식을 실무 도입안으로 정리해 팀에 제안하며 개인 프로젝트의 학습을 업무 개선으로 연결하려고 했습니다.

기술 스택

TypeScript

React

Next.js

SWR

Tanstack Query

Redux Toolkit

Zustand

styled-components

Tailwind CSS

shadcn/ui

Chakra UI

Turborepo

pnpm

Docker

GitHub Actions

AWS

openapi-typescript

NestJS

Prisma

Supabase

오픈놀 | Frontend Developer

2023.09 - 2025.09

교육·채용 플랫폼 미니인턴의 B2B, B2C, 어드민 프론트엔드 개발 및 유지보수를 담당했습니다.

TypeScript

Next.js

SWR

Redux Toolkit

MSW

styled-components

Turborepo

1. 세 도메인의 UI 컴포넌트 규격을 통합해디자인·개발 간 반복 확인을 줄인 디자인 시스템 구축

[세 도메인에서 달라진 UI 규격]

미니인턴 B2B·B2C·어드민은 서비스별로 UI 컴포넌트를 관리하고 있어 같은 역할의 버튼도 크기와 형태가 달랐습니다. 동일한 상태도 sm·md 또는 small·medium처럼 서로 다른 props로 표현했습니다.

서비스 요구에 따라 예외적인 형태가 추가되면서 Figma에 새로 정리된 디자인 시스템과 실제 구현에도 차이가 생겼고 새 화면을 만들 때마다 기존 버튼과 디자인 시안의 크기와 상태를 다시 비교해야 했습니다. 디자인 시스템의 적용 대상인지 예외인지도 디자이너에게 반복해서 확인했습니다.

[Headless UI 도입과 기존 컴포넌트 개선에 대한 논의]

구현 방식을 논의하며 Radix UI와 같은 Headless UI 라이브러리 도입을 제안했습니다. 드롭다운 기준으로 직접 구현할 경우 Portal을 이용한 렌더링과 위치 계산뿐 아니라 스크롤 대응·애니메이션·키보드 이동·포커스 관리·ARIA 속성같이 이미 만들어진것을 다시 만들 필요는 없다고 판단했습니다.

Headless UI는 접근성과 상호작용이 검증된 기능을 제공하고 기능적인 부분은 자체적으로 업데이트해주기 때문에 서비스 디자인에 맞는 스타일만 신경쓰면 되기 때문에 리소스도 줄어들고 공통 동작을 직접 다시 구현하는 부담도 줄일 수 있다고 판단했습니다.

팀에서는 기존 컴포넌트를 교체할 때 발생할 회귀 문제와 세 도메인의 화면을 다시 검증해야 하는 비용을 더 큰 위험으로 판단했고 논의 끝에 기존 컴포넌트를 기반으로 규격과 동작을 개선하기로 결정했습니다.

최종 결정에 따라 드롭다운의 기존 구조를 개선했습니다. 부모 요소의 overflow 속성으로 콘텐츠가 잘리는 문제는 React Portal로 해결했고 드롭다운이 열린 동안에만 스크롤·리사이즈 이벤트를 등록해 트리거와 콘텐츠의 위치를 동기화했습니다.

[사용처 조사로 공통화 기준 결정]

디자이너가 세 도메인의 모든 화면을 직접 확인하는 것 보다는 개발자가 코드에서 버튼 사용처와 예외 사례를 조사하는게 더 리소스가 적다고 판단해서 컴포넌트들의 사용처를 파악한 후 조사 결과를 함께 검토하며 공통 컴포넌트에서 제공할 크기와 상태 및 서비스별 예외로 남길 형태를 구분했습니다.

Figma 속성명과 코드 props를 같은 이름으로 맞춰 디자인 시안만 보고 사용할 컴포넌트를 선택할 수 있도록 했으며 공통 컴포넌트는 Turborepo의 packages/ui에 구성해 세 도메인에서 같은 규격으로 재사용할 수 있도록 했습니다.

[Storybook으로 검증하고 기존 화면 전체 전환]

버튼, 툴팁, 드롭다운의 크기와 상태 및 변형을 Storybook에서 한눈에 확인할 수 있도록 문서화하고 기본 상호작용 테스트를 작성했고 기존 화면에 적용하기 전에는 디자이너와 Storybook을 함께 검토해 전달받은 의도와 다르게 구현된 형태를 수정했습니다.

버튼은 세 도메인의 기존 화면 전체를 공통 컴포넌트로 전환했고 실제 화면에서는 수동 QA로 클릭 동작과 레이아웃을 확인하며 발견된 예외를 보완했습니다.

전환 이후에는 Figma 속성명만으로 사용할 컴포넌트를 선택할 수 있게 됐고 디자인 시스템과 다른 형태가 필요한 경우에는 Figma에 예외를 명시하도록 규칙을 정했습니다. 그 결과 화면마다 버튼의 크기와 스타일을 다시 확인하던 과정을 줄일 수 있었습니다.

[기술적 완성도와 전환 비용 사이에서 배운 점]

이번 경험을 통해 기술적으로 더 완성도 높은 선택이 항상 팀이 바로 적용할 수 있는 선택은 아니라는 점을 배웠습니다. Headless UI는 접근성과 유지보수 측면에서 장점이 있었지만 기존 기능의 안정성과 전환 범위 및 검증 비용도 함께 고려해야한다는걸 알게되었습니다.

디자인 시스템은 단순히 컴포넌트를 하나의 패키지로 모으는 것만으로 완성되는것이 아니라 디자이너와 개발자가 소통을 통해 컴포넌트별로 규칙 결정 및 문서화를 진행하고, 같은 속성을 사용하며, 공통 규격과 예외를 함께 관리할 수 있어야 실제 작업 방식까지 바꿀 수 있다는 점을 확인했습니다.

당시에는 기존 색상 토큰을 유지하면서 컴포넌트의 상태와 variant를 통합하는 데 집중해 타이포그래피·간격·모서리와 같은 파운데이션까지 충분히 정리하지 못한 점이 아쉬움으로 남았습니다. 다시 진행한다면 파운데이션을 먼저 정의하고 화면별 전환 체크리스트와 시각 회귀 테스트를 활용해 실제 화면에 미치는 영향까지 검증할 것입니다.

2. 무료 콘텐츠와 SEO 구조를 확장해 검색 유입을 유료 콘텐츠 진입으로 연결한 해피폴리오 개편

[두 차례의 스프린트로 이어진 서비스 개편]

해피폴리오는 사용자가 제작한 포트폴리오 PDF를 판매하는 미니언턴의 주요 수익 서비스였습니다. 오랫동안 운영된 서비스로 화면과 코드가 모두 낡아 있었으며 판매용 PDF를 제작하거나 구매하려는 사용자 외에는 서비스에 유입될 접점이 부족했습니다.

첫 번째 스프린트에서는 기존 유료 해피폴리오의 목록과 상세 화면 및 전체 레이아웃을 개편했습니다. 다음 스프린트에 무료 콘텐츠가 추가될 계획을 고려해 유료 콘텐츠에만 맞춰진 화면을 그대로 다시 만드는 대신 두 콘텐츠 유형을 함께 수용할 수 있도록 컴포넌트와 화면 구조를 정리했습니다.

두 번째 스프린트에서는 여러가지 분야에 대한 본인의 생각을 가볍게 작성하고 공유할 수 있는 무료 해피폴리오(스낵)를 추가했습니다. 검색을 통해 무료 콘텐츠에 유입된 사용자가 기존 유료 콘텐츠까지 탐색할 수 있도록 두 콘텐츠의 목록과 상세 흐름을 연결했습니다.

[유료·무료 콘텐츠를 수용하는 사용자 영역 구성]

유료·무료 콘텐츠의 목록과 상세 화면을 비롯해 검색·필터·북마크·내 콘텐츠 관리 영역을 담당했습니다. 구매와 결제는 안정적으로 운영되던 기존 흐름을 유지하고 개편된 목록과 상세 화면에서 해당 기능으로 연결했습니다.

데이터 요청은 팀에서 사용하던 SWR 기반으로 통일했습니다. 기존에는 DB 스키마 전체를 담은 하나의 큰 타입을 API 응답과 여러 컴포넌트의 props에 공통으로 사용하고 있었습니다. 실제 사용 필드를 기준으로 API 응답과 컴포넌트 props 타입을 분리하고 TypeScript 유틸리티 타입으로 공통 필드와 화면별 필드를 조합해 필요한 데이터만 의존하도록 정리했습니다.

목록·마이페이지·북마크에서 반복되는 콘텐츠 카드는 해피폴리오 도메인 내부의 공통 컴포넌트로 구성했습니다. 화면별 차이는 제한된 옵션으로 처리하고 조건 분기가 늘어나는 경우에는 별도 컴포넌트로 분리해 공통화로 인한 복잡도를 제한했습니다.

[기획자와 함께 정의한 SEO 노출 규칙]

무료 콘텐츠를 검색 유입으로 연결하려면 콘텐츠마다 제목·설명·공유 썸네일을 어떤 기준으로 노출할지 정해야 했습니다. 각 메타데이터의 역할과 검색 및 공유 화면에서의 차이를 기획자에게 설명하고 콘텐츠 유형별 제목과 설명 및 썸네일 규칙을 함께 정의했습니다.

Next.js의 동적 메타데이터 기능으로 콘텐츠별 제목·설명·썸네일을 서버에서 생성했습니다. 새 콘텐츠가 발행될 때 검색 엔진이 페이지를 발견할 수 있도록 동적 사이트맵도 구현했습니다.

개편 이후 무료 콘텐츠를 통한 검색 유입이 발생했고 일부 사용자는 유료 콘텐츠까지 진입했습니다. 기획자가 확인한 운영 결과를 통해 검색 유입이 실제 매출로 연결되는 흐름도 확인했습니다.

[연속된 스프린트에서의 범위와 안정성 관리]

첫 번째 스프린트에서 기존 화면을 개편하면서 공용 타입과 컴포넌트가 여러 화면에 얹혀 있어 변경 범위가 예상보다 크다는 점을 확인했습니다. 모든 구조를 한 번에 다시 설계하기보다 첫 번째 스프린트에서는 기존 유료 서비스의 안정성과 다음 기능을 수용할 화면 구조를 우선했습니다.

두 번째 스프린트에서는 앞서 정리한 구조를 바탕으로 무료 콘텐츠 영역과 SEO 기능을 추가했습니다. 구매·결제처럼 안정적으로 운영되던 기능은 변경하지 않고 새 콘텐츠 흐름과 연결해 회귀 위험을 줄였습니다.

스프린트 방식으로 전환 후 개발시간적 여유가 부족하여 팀과 논의 후 자동화 테스트보다 핵심 기능 구현과 수동 검증을 우선했습니다. 개발을 하루에서 이틀 먼저 마무리한 뒤 개발자와 기획자가 주요 사용자 흐름을 함께 확인했으며 기존 구매·결제 시나리오도 회귀 테스트해 큰 출시 장애 없이 두 차례의 스프린트를 마무리했습니다.

[레거시 개편을 통해 달라진 일정 산정 기준]

기존 화면을 개선하는 작업이라 신규 기능보다 짧게 끝낼 것으로 예상했지만 실제로는 공용 타입과 컴포넌트의 결합으로 인해 작은 변경도 여러 화면에 영향을 줄 수 있었습니다. 일정 안에서 두 차례의 개편을 마무리하기 위해 타입과 컴포넌트 구조는 필요한 범위까지만 개선했습니다.

이 경험을 통해 레거시 작업은 코드를 읽는 것만으로 변경 범위를 모두 예측하기 어렵다는 점을 배웠습니다. 이후에는 공용 코드의 참조 범위와 영향 화면을 먼저 확인하고 불확실성과 회귀 검증 시간을 포함해 일정을 보수적으로 산정했습니다.

앞으로 다른 레거시 개선 작업을 진행한다면 수정 전에 영향받는 화면과 공용 코드를 목록화하고 기능 개편과 구조 개선의 범위를 분리해 예상 범위와 실제 범위의 차이를 줄일 것입니다.

3. 스킵업 서비스와 공통 네비게이션 가드를 App Router로 전환한 마이그레이션

[기능 동등성을 우선한 전환 범위 결정]

Pages Router의 유지보수 단계 진입에 대비하고 Server Components와 Streaming 같은 신규 기능을 활용할 기반을 마련하기 위해 App Router 전환을 진행했습니다.

기존 프로젝트는 styled-components와 클라이언트 상태를 사용하는 컴포넌트가 많았습니다. 마이그레이션과 동시에 서버·클라이언트 경계와 레이아웃 구조까지 다시 설계하면 변경 범위와 회귀 위험이 커질 수 있다고 판단했습니다.

첫 단계에서는 새로운 렌더링 기능을 적용하기보다 기존 동작을 유지하는 데 집중했습니다. page 컴포넌트가 서버 데이터 요청을 담당하도록 변경하되 상태와 상호작용을 포함한 UI는 기존 Client Component 구조를 유지했습니다. Server Components와 Streaming을 활용한 최적화는 기능 동등성 확보 이후의 과제로 분리했습니다.

[화면 특성과 API 구조를 고려한 데이터 패칭 전환]

강의 목록은 등록 빈도가 낮고 일정 수준의 갱신 지연을 허용할 수 있었습니다. 기존 getStaticProps의 revalidate로 구현했던 ISR을 Server Component의 fetch와 next.revalidate 옵션으로 전환해 정적 렌더링과 주기적인 데이터 갱신 방식을 유지했습니다.

강의 상세 API는 공통 강의 정보와 사용자별 구매 상태가 분리돼 있지 않았습니다. 로그인 쿠키에 따라 구매하기 또는 시청하기 상태를 함께 반환했기 때문에 공통 데이터만 서버에서 캐시하기 어려웠습니다. 기존 API 계약을 변경하는 대신 쿠키를 포함한 SWR 요청을 유지해 클라이언트에서 상세 정보와 구매 상태를 함께 조회했습니다.

내 강의 화면도 로그인 사용자에게만 필요한 데이터이므로 SWR 기반 CSR을 유지했습니다. 모든 화면을 일괄적으로 Server Component로 변경하지 않고 데이터의 갱신 주기와 개인화 여부 그리고 기존 API 구조를 기준으로 렌더링 방식을 선택했습니다.

[공통 네비게이션 가드의 App Router 대응]

여러 단계에 걸쳐 내용을 작성하는 폼에서는 사용자가 입력 도중 페이지를 벗어날 경우 작성 데이터가 사라질 수 있었습니다. 기존에는 Pages Router의 router.events를 구독하는 공통 네비게이션 가드로 이탈을 감지했지만 App Router에서는 해당 이벤트가 제거돼 기존 방식을 사용할 수 없었습니다.

화면 내부의 단계 이동은 작성 상태와 현재 단계를 기준으로 제어했습니다. 브라우저의 뒤로가기와 앞으로가기는 popstate와 History API를 조합해 처리했으며 새로고침과 탭 종료는 beforeunload로 감지했습니다.

가드가 활성화된 상태에서는 현재 경로를 history stack에 보존했습니다. 브라우저 이동이 발생하면 이탈 여부를 확인하고 사용자가 취소할 경우 현재 경로를 복원해 작성 중인 데이터를 보호했습니다.

[브라우저 히스토리의 한계와 지원 범위 결정]

브라우저 뒤로가기는 popstate를 이용해 이벤트를 등록할 수 있지만 먼저 막는게 아니라 히스토리가 이동한 뒤 발생하므로 이탈을 막기위해서 히스토리에 강제로 현재 위치를 하나 더 넣는 방식으로 제어했습니다. 다만 유저가 URL 직접 변경과 새로고침 그리고 브라우저 버튼과 화면 내부 버튼을 반복해서 사용하면 브라우저 히스토리가 중복되거나 잘못된 단계에서 이탈 확인 창이 나타나는 문제가 있었습니다.

모든 이동 조합을 방어하면 공통 가드의 복잡도가 커지고 기존 사용 화면에도 영향을 줄 수 있었습니다. 기획자와 논의한 뒤 단계 이동과 이전 단계 수정 그리고 최종 확인과 완료로 이어지는 주요 사용자 흐름을 우선 보장했습니다. 여러 이동 방식을 의도적으로 반복하는 비정상적인 조합은 지원 범위에서 제외했습니다.

[마이그레이션 과정에서 정립한 판단 기준]

프레임워크 전환같은 대규모 마이그레이션에서는 새로운 기능을 한 번에 적용하기보다는 기존 동작을 어느 범위까지 보장할지 먼저 정하는 일이 중요하다는 점을 배웠습니다. 기존 동작을 모두 확인 후 새로운 기능을 적용하는 방향으로 진행하는게 좋은 방향이라는걸 알게 되었습니다.

데이터 패칭도 Server Component 사용 자체를 목표로 삼기보다 데이터의 갱신 주기와 개인화 여부 그리고 API 구조를 함께 고려해야 했습니다. 네비게이션 가드는 브라우저 이동을 완전히 통제하는 기능이 아니므로 지원할 사용자 흐름을 먼저 정의하고 시나리오별 히스토리 변화를 검증해야 한다는 점도 확인했습니다.

4. 검증 기반이 부족한 관리자 레거시를 작업 범위부터 SWR로 점진 전환

[데이터 흐름을 파악하기 어려웠던 관리자 구조]

관리자 페이지는 Pages Router의 SSR과 Redux Saga를 기반으로 구현돼 있었습니다. API 요청 하나에도 action과 saga 그리고 reducer를 거쳐야 했으며 서버에서 가져온 데이터도 전역 상태로 관리했기 때문에 단방향 구조가 아니어서 데이터 흐름을 추적하는게 힘들었습니다.

기술 도입의 배경을 확인한 결과 관리자 서비스의 요구사항에 따른 선택이라기보다 당시 개발 과정에서 도입한 구조에 가까웠습니다. 내부 사용자만 접근하는 화면이었고 데이터도 로그인 이후에 조회했기 때문에 모든 화면에 SSR을 유지해야 할 필요성이 크지 않았습니다.

API 요청이 여러 파일에 나뉘어 있어 화면 코드만으로 호출 위치와 데이터 변경 과정을 파악하기 어려웠습니다. 관련 코드를 추적한 뒤 로그를 확인해야 실제 흐름을 확신할 수 있었으며 작은 기능을 수정할 때도 탐색 비용이 반복해서 발생했습니다.

[전면 교체 대신 작업 범위부터 시작한 점진적 전환]

관리자 전체를 한 번에 전환하면 구조를 일관되게 만들 수 있었지만 테스트 코드와 요구사항 이력이 남아 있지 않았습니다. 기존 기획을 설명할 수 있는 담당자도 없어 변경 이후의 동작을 자동으로 검증하기 어려운 상황이었습니다.

전면 교체에 따른 회귀 위험이 개선 효과보다 크다고 판단해 신규 기능이나 리팩토링이 예정된 화면만 전환했습니다. 기존 코드를 그대로 유지해야 하는 영역과 이번 작업에서 동작을 확인할 수 있는 영역을 나누고 검증 가능한 범위 안에서 레거시를 줄였습니다.

한 번의 대규모 마이그레이션을 진행했을때 사이드 이펙트가 어디서 발생할지 아무도 히스토리를 알지 못하는 상태였기 때문에 작업 계획된 영역이 발생하면 이 방식으로 화면을 순차적으로 변경했으며 기능 작업과 레거시 개선을 함께 진행했습니다.

[SSR·Redux Saga에서 SWR로 데이터 요청 단순화]

SSR로 전달하던 데이터는 화면의 사용 방식과 갱신 시점을 다시 확인한 뒤 클라이언트 요청으로 전환했습니다. Redux Saga에서 관리하던 서버 상태는 팀에서 사용하던 SWR로 옮겨 API 요청과 캐시 그리고 재검증 흐름을 화면 가까이에서 확인할 수 있도록 했습니다.

기존에는 하나의 API 요청을 위해 action과 saga 그리고 reducer를 각각 작성해야 했습니다. 전환 이후에는 화면에서 SWR 혹은 통해 요청과 응답을 바로 확인할 수 있어 관련 보일러플레이트를 제거했습니다.

관리자 화면도 기존 구직자 서비스와 같은 데이터 패칭 방식을 사용하게 됐습니다. 동료 개발자로부터 이전보다 데이터 요청 위치와 코드 흐름을 읽기 쉬워졌다는 피드백을 받았습니다.

[기획자와 영향 범위를 확인한 수동 검증]

자동화된 테스트가 없었기 때문에 변경 전에 관리자 화면과 사용자 서비스 사이의 영향을 먼저 확인했습니다. 관리자에서 수정한 데이터가 사용자 화면의 어느 영역에 노출되는지 기획자와 개발자가 함께 파악했습니다.

레거시 구조를 변경하면서 현재 작업 범위 밖에도 영향이 발생할 수 있다는 점을 기획자에게 공유했습니다. 개발 단계에서는 주요 기능을 직접 확인했고 기획자에게는 연결된 사용자 화면까지 함께 검토해 달라고 요청했습니다.

기능 추가와 기존 흐름의 회귀 여부를 수동으로 확인한 뒤 배포했으며 전환으로 인한 운영 문제는 발생하지 않았습니다.

[레거시 개선에서 정립한 범위 판단]

레거시는 불편하다는 이유만으로 전체를 교체하기보다 현재 검증할 수 있는 범위를 기준으로 개선해야 한다는 점을 배웠습니다. 구조적으로 더 나은 코드로 변경하는 것만큼 수정 후 동작을 검증할 수 있는 방법에 대해 정해두는것도 중요하다는것을 알게 되었습니다.

다시 진행한다면 수정 전에 공용 타입과 컴포넌트의 참조 범위를 목록화하고 영향받는 화면을 검증 항목으로 만들고, 레거시 작업의 일정도 코드 수정 시간만으로 산정하지 않고 영향 범위 파악과 회귀 검증에 필요한 시간을 별도로 반영할 것입니다.

jabaek | FullStack Project

2026.07 - 진행중

Service (PlayStore) | GitHub (Private Repo)

5명의 AI 용의자를 심문하고 증거와 진술의 모순을 추적해 진범을 찾는 웹·모바일 머더 미스터리 게임

TypeScriptNext.jsTanStack QueryReact NativeExpoZodSupabaseTurborepo

ClaudeGeminiCodex

1. AI와 협업해 기획부터 Google Play 출시까지 완성한 1인 개발

[프로젝트 목표와 담당 범위]

AI 시대의 개발자가 제품을 어디까지 설계하고 검증해야 하는지 확인하기 위해 시작한 1인 프로젝트입니다.

초기에는 AI가 생성한 사건 속 용의자들을 심문하며 범인을 찾는다는 아이디어만 가지고 있었습니다. 이후 AI와 대화하며 게임 규칙과 사용자 흐름을 구체화했고 기술 선택과 아키텍처 및 검증 기준을 직접 결정했습니다. Codex를 활용해 코드와 테스트 및 문서를 구현하고 웹 서비스와 Android 앱을 출시했습니다.

[역할과 비용을 고려한 AI 모델 분리]

하나의 모델에 모든 역할을 맡기지 않고 작업의 특성에 따라 AI를 분리했습니다.

논리적인 인물 관계와 증거 및 진술을 구성해야 하는 사건 생성에는 Claude를 사용했습니다. 플레이 중 반복적으로 호출되는 용의자 대화에는 주어진 역할과 지식 범위 안에서 응답하는 Gemini를 적용해 비용을 고려했습니다. Codex는 제품 정책과 기술 구조를 바탕으로 코드와 테스트 및 문서를 구현하는 개발 도구로 사용했습니다.

AI가 서비스의 상태를 직접 결정하지 않도록 책임 경계도 분리했습니다. 외부 AI 응답은 Zod로 검증하고 결제 권한과 사건의 정답 및 상태 전이는 도메인 코드가 최종 판정하도록 구성했습니다.

[웹과 모바일을 연결한 서비스 출시]

Next.js로 웹 서비스와 서버 기능을 구성하고 Expo WebView로 Android 앱을 구현했습니다.

웹과 네이티브 사이에는 web_loading → bridge_ready → interactive 상태를 두어 브리지 준비 전 요청이나 중복 메시지가 화면 상태를 덮어쓰지 않도록 제어했습니다.

Google Play 결제는 클라이언트의 성공 응답만으로 권한을 지급하지 않고 서버 검증 이후 반영했습니다. 알림은 생성 상태와 전송 상태를 분리해 실패한 전송만 재시도할 수 있게 했으며 Sentry와 source map을 연결해 웹과 모바일 오류를 추적했습니다.

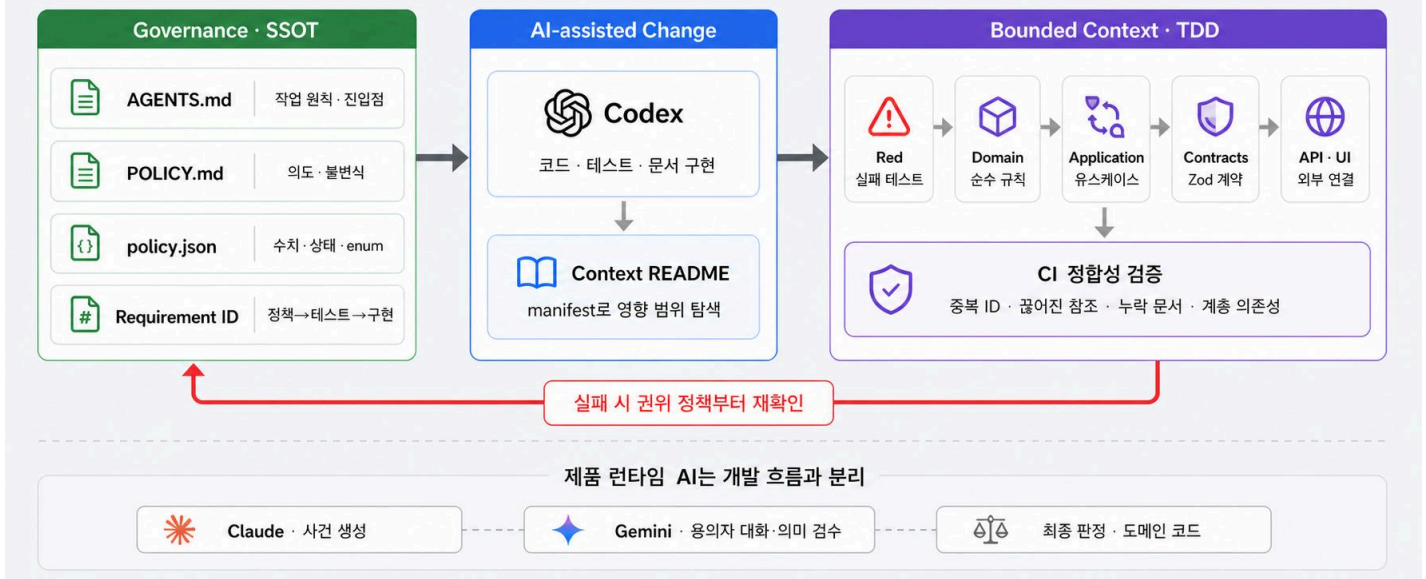
[결과와 배움]

사건 생성과 플레이 및 결제와 알림이 하나의 서비스 흐름으로 동작하는 제품을 Google Play에 출시했습니다.

AI를 사용하면 코드 작성 속도는 빨라지지만 제품의 정책과 책임 경계까지 자동으로 일관되게 유지되지는 않았습니다. 이 경험을 통해 개발자의 역할이 코드를 직접 작성하는 데서 끝나지 않고 AI가 따라야 할 기준을 정의하며 결과를 검증하고 출시 이후까지 책임지는 일로 확장된다는 점을 배웠습니다.

AI 협업의 정책·코드 드리프트를 줄인 변경 흐름

권위 정책 확인 → 실패 테스트 작성 → Context 내부 구현 → 문서·테스트·코드 정합성 검증



2. AI 협업의 정책·코드 드리프트를 줄인 SSOT·Bounded Context·TDD 체계

[AI가 빠르게 만들수록 커진 불일치]

여러 AI와 반복해서 작업하면서 구현 속도는 빨라졌지만 변경을 요청할 때마다 새로운 문서와 중복 타입 및 기존 정책과 충돌하는 규칙이 생겼습니다.

특히 결제와 사건 생성 및 플레이 상태처럼 여러 영역에 걸친 정책은 일부 문서와 테스트 및 코드만 변경되기 쉬웠습니다. 같은 개념을 서로 다른 기준으로 해석하면 당장은 동작하더라도 다음 작업에서 어떤 규칙이 맞는지 다시 판단해야 했고 AI도 오래된 구현을 새로운 정책처럼 참조했습니다.

[정책의 권위 원본과 변경 범위 정의]

같은 정책을 여러 문서에 반복하지 않고 각 규칙의 권위 원본을 하나로 지정했습니다. 사람이 이해해야 하는 정책의 의도와 불변식은 POLICY.md에 두고 코드에서 판정할 숫자와 enum 및 상태는 policy.json에서 관리했습니다.

계정과 결제 및 사건과 플레이 같은 영역은 Bounded Context로 분리했습니다. AGENTS.md는 작업의 공통 원칙과 문서 진입점을 제공하고 docs/manifest.json은 변경할 Context와 권위 문서를 찾는 라우터 역할을 담당하도록 구성했습니다.

[정책에서 구현까지 이어지는 변경 순서]

모든 변경은 요구사항 → 권위 정책 → 실패 테스트 → Domain · Application · Contracts → API · UI 순서로 진행했습니다.

각 규칙에는 Requirement ID를 부여해 정책과 테스트 및 구현 위치를 연결했습니다. AI가 작업을 시작하기 전에 영향받는 Context와 권위 문서를 확인하게 하고 정책을 수정한 뒤 해당 Requirement ID를 가진 가장 작은 실패 테스트부터 작성하도록 했습니다. 이후 도메인 규칙과 유스케이스 및 외부 계약을 구현하고 마지막에 API와 UI를 연결했습니다.

[자동화한 문서와 코드의 정합성 검증]

GitHub Actions에서 중복된 Requirement ID와 끊어진 참조 및 누락된 권위 문서를 검사했습니다. 계층 간 허용되지 않은 의존성과 정책에서 생성된 문서의 불일치도 함께 차단했습니다.

이 검사는 테스트의 성공 여부만 확인하는 것이 아니라 정책과 테스트 및 구현이 같은 요구사항을 가리키는지 검증합니다. 따라서 AI가 별도의 정책 문서나 중복 타입을 추가하더라도 변경 단계에서 발견할 수 있습니다.

[변경 비용과 일관성 사이의 선택]

초기에 정책과 요구사항의 연결 구조를 만드는 비용은 단순 구현보다 컸습니다. 작은 규칙까지 모두 문서화하면 오히려 유지 비용이 늘어날 수 있어 오류의 영향이 큰 정책은 엄격하게 추적하고 단순한 UI 세부사항은 코드와 테스트에 맡기는 기준을 세웠습니다.

이 구조가 모든 오류를 제거하지는 않았지만 AI가 작업을 시작하기 전에 변경 범위를 확인하고 작업이 끝난 뒤 정책부터 구현까지의 누락을 검사할 수 있게 했습니다. 여러 AI 세션을 거치더라도 기존 기준을 이어받을 수 있는 개발 환경을 만들었다는 점이 가장 큰 결과입니다.

story-dict | FullStack Project

2024.10 - 2025.01

[GitHub](#) | [API Docs](#)

일상에서 이름이 떠오르지 않는 단어나 표현을 질문하고 지식을 공유하는 커뮤니티 서비스

> 기술 검증과 풀스택 아키텍처 학습을 목적으로 진행한 프로젝트로, 계획한 개발 목표를 달성한 후 인프라 유지 비용을 고려해 운영을 종료했습니다

TypeScript

Next.js

Tanstack Query

NestJS

Prisma

openapi-typescript

Tailwind CSS

shadcn/ui

Turborepo

Docker

AWS

GitHub Actions

1. 수동 API 문서의 불일치를 줄인 OpenAPI 기반 계약 자동화

[실무에서 발견한 문서와 응답의 차이]

실무에서는 Postman으로 API 문서를 관리했고 백엔드 개발자가 변경 내용을 수동으로 반영했습니다. 이 과정에서 문서에는 string으로 적혀 있지만 실제 응답에는 null이 포함되는 것처럼 문서와 코드의 계약이 달라지는 일이 반복됐습니다.

프론트엔드에서는 문서만 신뢰하기 어려워 실제 응답이나 백엔드 코드를 다시 확인해야 했습니다. 변경된 필드가 타입에 반영되지 않으면 구현 이후에야 오류를 발견할 수 있었고 API가 늘어날수록 확인할 범위도 함께 커졌습니다.

[API 계약 자동화 방식 비교]

문서와 실제 응답의 불일치를 줄이기 위해 세 가지 방식을 비교했습니다. 먼저 tRPC는 프론트엔드와 백엔드가 모두 TypeScript를 사용하고 모노레포로 코드를 공유해야 했습니다. 당시 백엔드는 TypeScript 기반으로 완전히 전환되지 않았고 기존 저장소를 모노레포로 바꾸기도 어려워 적용하지 않았습니다.

백엔드 타입을 별도 패키지로 만들어 NPM에 배포하는 방식도 검토했습니다. 하지만 타입이 변경될 때마다 패키지를 배포하고 프론트엔드와 백엔드의 버전을 관리해야 해 백엔드 리소스와 운영 부담이 컸습니다.

최종적으로 기존 REST API 구조를 유지하면서 백엔드 코드를 기준으로 명세를 생성할 수 있는 OpenAPI 방식을 선택했습니다.

[백엔드 코드에서 OpenAPI 명세 생성]

NestJS의 DTO와 Controller에 선언된 타입 및 검증 정보를 @nestjs/swagger가 읽도록 구성했습니다. SwaggerModule로 OpenAPI 문서를 생성하고 개발 환경에서 YAML 파일로 저장해 백엔드 코드의 변경이 명세에 반영되도록 했습니다.

문서를 별도로 작성하는 대신 실제 API 코드에서 데코레이터 기반으로 문서가 작성되는 구조를 만들었고 데코레이터와 같은 타입을 사용하여 백엔드의 요청과 응답 정의가 OpenAPI 명세에 자동으로 반영되도록 구성했습니다.

[openapi-typescript로 프론트엔드 타입 자동 생성]

OpenAPI 명세에서 TypeScript 타입을 생성하는 openapi-typescript 라이브러리를 사용했습니다. 생성된 YAML을 변환해 프론트엔드의 paths와 components 타입을 자동 생성하고 openapi-fetch와 openapi-react-query를 연결해 타입이 적용된 TanStack Query 요청 계층을 구성했습니다.

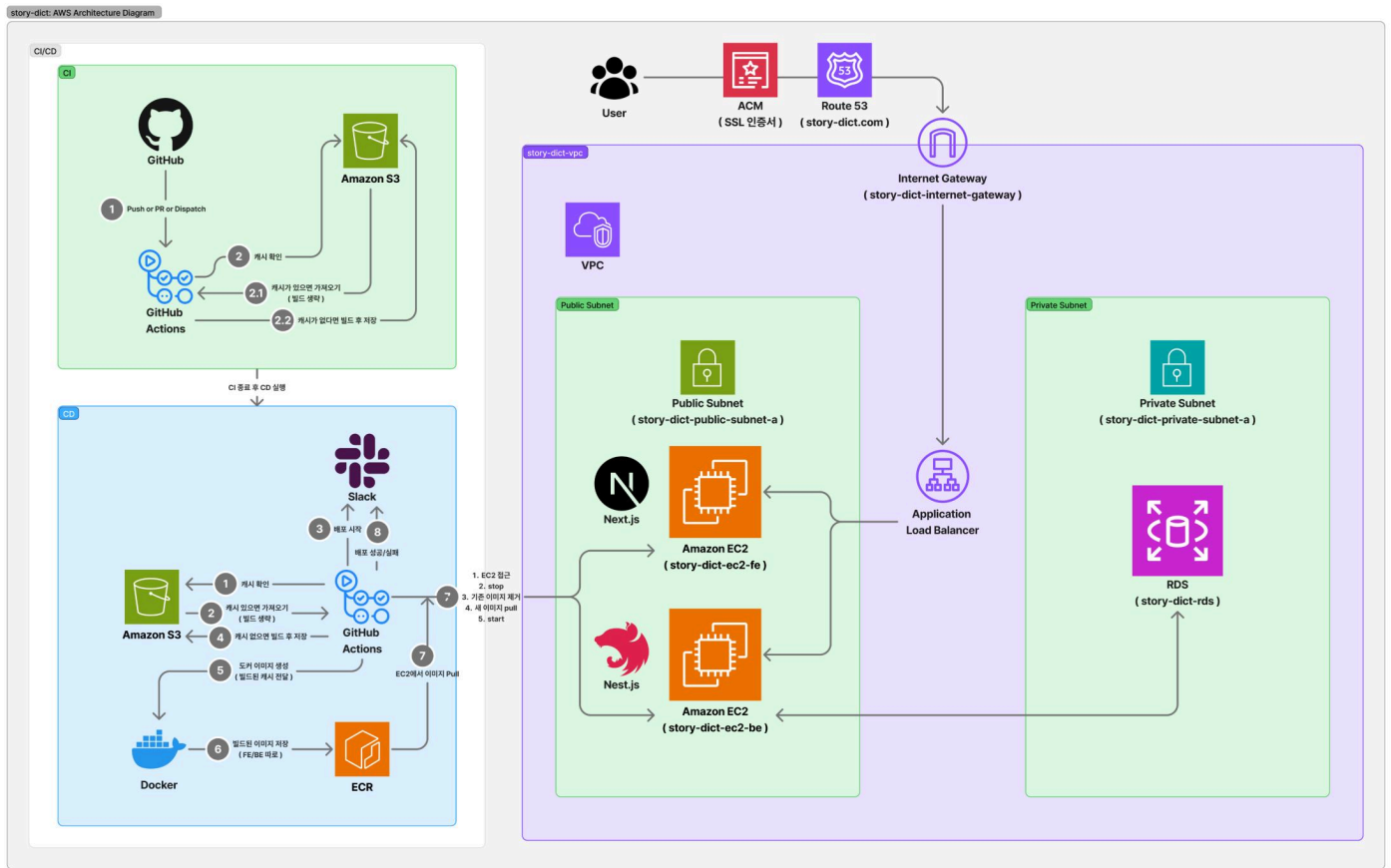
엔드포인트나 nullable 필드가 변경되면 프론트엔드 타입 오류로 드러나도록 만들었습니다. 개발자가 응답 타입을 별도로 선언하거나 백엔드 문서를 다시 옮겨 적는 과정을 줄이고 코드와 명세 및 프론트엔드 타입이 같은 계약을 참조하게 했습니다.

[개인 프로젝트 검증에서 실무 제안까지]

현 프로젝트의 게시글과 댓글 및 사용자 API에 이 구조를 적용하며 기존 REST API에서도 명세를 중심으로 타입 안전성을 높일 수 있음을 확인했습니다.

검증한 방식과 적용 조건을 정리해 실무 도입안으로 제안했지만 이 부분은 결국 DX를 위한 개선 작업이고 현재 이것보다 선행으로 개발해야 할 작업들이 많아서 백로그로 남겨두게 되었습니다. 프론트 작업은 제가 시간을 내서 할 수 있었지만 백엔드에서 해야 하는 일이 많아서 진행하지 못하게 되었습니다.

단순히 타입을 꼼꼼히 작성하는 것만으로는 계약 불일치를 막기 어렵고 어떤 원본에서 타입을 파생할지 정하는 것이 중요하다는 점을 배웠습니다. 이후에는 새로운 기술을 선택할 때 현재 구조와 전환 비용 및 운영 부담을 함께 생각하게 되었습니다.



2. Docker와 AWS 인프라를 직접 구성해 구축한 프론트엔드·백엔드 CI/CD

[배포 과정 전체를 이해하기 위한 인프라 구성]

완성된 플랫폼에 서비스를 올리는 수준을 넘어 사용자의 요청이 애플리케이션과 데이터베이스에 도달하는 흐름을 직접 구성하고 싶었습니다.

VPC 안에 프론트엔드와 백엔드 EC2를 배치하고 RDS는 Private Subnet에서 외부에 직접 노출되지 않도록 구성했습니다. 보안 그룹으로 애플리케이션과 데이터베이스 사이의 접근 범위를 제한했으며 Route 53과 ACM 및 Application Load Balancer를 연결해 도메인과 HTTPS 요청을 처리했습니다. 게시글 이미지는 Presigned URL을 통해 S3에 업로드하도록 구현했습니다.

[Docker 이미지 기반의 자동 배포]

프론트엔드와 백엔드를 각각 Docker 이미지로 구성하고 Turborepo의 prune을 활용해 서비스별 빌드에 필요한 워크스페이스만 포함했습니다.

GitHub Actions에서 소스 변경 범위를 확인한 뒤 필요한 이미지만 빌드해 ECR에 업로드했습니다. 이후 EC2가 해당 이미지를 받아 컨테이너를 교체하도록 구성해 빌드와 배포 및 실행 과정을 자동화했습니다. 각 단계의 성공과 실패 결과는 Slack 알림으로 확인할 수 있게 했습니다.

[Turborepo와 S3를 활용한 빌드 결과 재사용]

CI가 실행될 때마다 모든 워크스페이스를 다시 빌드하지 않도록 Turborepo의 의존성 그래프와 캐시를 활용했습니다. GitHub Actions 캐시로 .turbo 결과를 재사용하고 소스 해시가 같은 빌드 결과는 S3에서 복원하도록 구성했습니다.

CD에서도 동일한 소스 해시의 ECR 이미지가 존재하면 새 이미지를 만들지 않고 기존 이미지를 배포했습니다. 이를 통해 변경되지 않은 프론트엔드나 백엔드의 중복 빌드를 피하고 필요한 서비스만 배포하는 흐름을 만들었습니다.

[인프라를 직접 운영하며 얻은 이해]

Docker와 AWS 서비스를 개별 기능으로 이해하는 데서 그치지 않고 네트워크와 인증서 및 이미지 저장소와 실행 환경이 하나의 배포 과정에서 어떻게 연결되는지 확인했습니다.

초기에는 캐시와 배포 단계를 직접 구성하는 데 시간이 들었지만 실제 운영 환경에서는 빠른 빌드보다 캐시의 재사용 조건과 배포 대상 및 실패 지점을 명확히 만드는 일이 중요하다는 점을 배웠습니다.

blog | Frontend Project

2024.01 - 2024.07

Service | [GitHub](#)

기존 블로그 플랫폼의 표현과 확장성 한계를 해결하기 위해 직접 구축한 기술 블로그

TypeScript

Next.js

MDX

Tailwind CSS

shadcn/ui

Vercel

SEO와 표현력, 확장성을 고려한 MDX 기반 개인 기술 블로그 구축

[문제]

기존에 사용하던 [Velog](#)는 구조와 UI 커스터마이징에 한계가 있어 원하는 기능과 디자인을 자유롭게 반영하기 어려웠습니다.

또한 [GitHub Pages](#) 기반 블로그는 게시글 수가 늘어날수록 빌드 시간이 길어지고 성능이 저하되는 문제가 있었고, 검색 유입을 고려한 SEO 대응이나 게시글 표현 방식 확장에도 제약이 있었습니다.

결국 콘텐츠를 쌓아갈수록 운영 도구가 발목을 잡는 구조라고 느꼈고, 디자인부터 배포 방식까지 직접 통제할 수 있는 블로그를 따로 구축하기로 했습니다.

[접근]

블로그는 검색 유입과 게시글 로딩 속도가 중요하다고 판단해 Next.js App Router 기반의 SSG 구조를 선택했습니다. 빌드 시점에 게시글 페이지를 정적으로 생성하면 빠른 응답 속도와 SEO 친화적인 HTML을 확보할 수 있고, App Router의 파일 기반 구조를 활용하면 게시글 페이지와 메타데이터 처리도 일관되게 관리할 수 있다고 봤습니다.

콘텐츠 작성 방식은 MDX를 선택했습니다. 순수 마크다운보다 표현력이 높고 React 컴포넌트를 문서 안에 직접 조합할 수 있어, 코드 하이라이팅, 콜아웃, 이미지 캡션 같은 UI를 재사용 가능한 형태로 확장할 수 있기 때문입니다.

또한 초기에는 누구나 저장소를 포크해 자신의 블로그로 사용할 수 있는 구조를 목표로 했기 때문에, 외부 서비스나 데이터베이스 의존을 최대한 줄이려 했습니다. 그래서 게시글과 메타데이터는 모두 Git 기반 파일 구조로 관리하고, 별도의 관리자 페이지나 DB 없이 운영할 수 있는 형태로 설계했습니다.

[해결]

게시글 페이지는 App Router 기반 SSG로 구성해 빌드 시점에 정적 페이지가 생성되도록 했습니다. MDX를 도입해 마크다운의 제한적인 표현 방식을 보완했고, 코드 하이라이팅, 콜아웃, 이미지 캡션 등 블로그에서 자주 사용하는 요소를 컴포넌트로 분리해 포스팅 안에서 일관되게 재사용할 수 있도록 구성했습니다.

또한 MDX front matter와 generateMetadata, sitemap, robots를 연동해 게시글별 메타데이터 생성과 검색 엔진 크롤링 설정을 자동화했습니다. 새 글이 추가되면 별도 SEO 작업 없이도 메타 정보와 사이트맵이 함께 반영되도록 만들었습니다.

운영 방식도 단순하게 유지했습니다. 관리자 화면 없이 Git 기반 콘텐츠 구조를 택하고, Vercel 자동 배포와 연결해 MDX 파일을 추가하면 포스팅이 바로 배포되도록 구성했습니다. 댓글 기능은 외부 서비스나 GitHub Issues 연동도 검토했지만, 블로그 디자인과 사용 흐름에 자연스럽게 녹지 않는다고 판단해 제외했습니다.

[성과]

블로그에 필요한 타임라인, 태그, 검색, 목차, 코드 하이라이팅, 다크 모드, 반응형 UI를 직접 구현하면서 기존 플랫폼의 제약 없이 원하는 기능과 디자인을 자유롭게 확장할 수 있는 구조를 만들었습니다.

또한 SSG와 메타데이터 자동화 구조를 통해 빠른 게시글 로딩과 검색 엔진 노출을 함께 고려할 수 있었고, 콘텐츠 작성부터 배포까지를 MDX 파일 중심으로 단순화해 운영 비용도 줄일 수 있었습니다.

무엇보다 외부 서비스와 데이터베이스 의존을 최소화한 덕분에, 개인 블로그를 단순히 “운영하는 사이트”가 아니라 필요하면 다른 사람도 포크해 재사용할 수 있는 구조로 설계할 수 있었습니다.

[느낀점]

콘텐츠 플랫폼은 단순히 글을 올리는 공간이 아니라, 작성 경험, 검색 노출, 성능, 표현력, 운영 방식까지 함께 설계해야 한다는 점을 배웠습니다.

또한 모든 기능을 다 넣기보다, 어떤 기능은 의도적으로 포기하는 판단도 중요하다는 점을 느꼈습니다. 댓글처럼 구현 가능한 기능이 있더라도 서비스의 방향성과 디자인 흐름에 맞지 않으면 넣지 않는 편이 더 나을 수 있다는 점이 인상적이었습니다.